

A 26-WEEK CURRICULUM

The Staff-Level Leap

From Senior Data Scientist to Staff Applied Scientist
Experimentation | Modern RecSys | GenAI | Leadership

Prepared for: Akash Singh

Senior Data Scientist, Mastercard Services | Targets: UAE · India · Netherlands

July 2026 – December 2026 | 15–20 hours per week

"The best way to predict your career is to build it, one deliberate week at a time."

Contents

A dependency-ordered curriculum: each chapter assumes only what came before it.

Front Matter

- A Letter From Your Mentor · The Learning Philosophy · How To Use This Book · Your Starting Position

Part I — Foundations That Compound (Weeks 1–8)

- Chapter 1 · Experimentation and A/B Testing at Scale (Weeks 1–2)
- Chapter 2 · Causal Inference (Weeks 3–4)
- Chapter 3 · PyTorch and Deep Learning Foundations (Weeks 5–8, parallel track)
- Chapter 4 · Modern Recommender Systems (Week 5)
- Chapter 5 · Learning to Rank (Week 6)
- Chapter 6 · Transformers From First Principles (Week 7)
- Chapter 7 · RAG Fundamentals (Week 8)

Part II — The GenAI Stack (Weeks 9–18)

- Chapter 8 · LLM Internals and Fine-Tuning: LoRA / QLoRA (Weeks 9–10)
- Chapter 9 · LLM and RAG Evaluation (Weeks 10, 14)
- Chapter 10 · Agentic AI Systems (Weeks 11–13)
- Chapter 11 · Production GenAI and ML System Design (Weeks 15–18)

Part III — The Portfolio (Weeks 9–18)

- Chapter 12 · Strategy and the Four Projects, with weekly milestones

Part IV — Interview Mastery and the Campaign (Weeks 19–26)

- Chapter 13 · SQL and Coding Under Pressure
- Chapter 14 · ML and AI System Design Interviews
- Chapter 15 · Behavioral Interviews and Staff-Level Leadership
- Chapter 16 · The Application Campaign: Referrals, Sequencing, Negotiation

Part V — The 26-Week Calendar

- Chapter 17 · The Week-by-Week Master Calendar with phase exit criteria

Appendices

- A · The Master Resource Library
- B · The Resume Rewrite Checklist
- C · Monthly Checkpoints and Self-Assessment
- D · The Forgotten-Math Refresher · A Closing Word

A Letter From Your Mentor

Read this first. Then read it again in Week 13 when you are tired.

Akash, I have read your resume and the gap analysis line by line. Here is my honest read, the way I would give it to a student sitting across from me at a whiteboard.

You are not behind on talent. You are behind on positioning and on three specific bodies of knowledge. You have shipped real ML at real scale — a VAE recommender that moved \$5M, a GPT-4o retrieval system that physicians actually use, a ranking model over a billion household-item pairs. Most candidates chasing Staff roles have never done any of that. What you are missing is (1) the statistical machinery of experimentation and causal inference that companies like Booking.com and Uber use as their primary hiring filter, (2) the modern architecture vocabulary — Two-Tower, LambdaMART, Transformers, LoRA, RAG, agents — that signals you live in 2026 and not 2021, and (3) the visible evidence of leadership and depth: a portfolio, stories, numbers, and opinions on trade-offs.

The good news: all three are learnable in 26 weeks at 15–20 hours per week, because you are not learning from zero — you are attaching new vocabulary and rigor to systems you have already built. When you learn Two-Tower models, you will recognize what your Wide & Deep system was trying to be. When you learn interleaving and position-bias correction, you will see your Albertsons CTR lift with new eyes. This is why an experienced engineer can compress what takes a fresh graduate three years into six months.

One more thing, because you mentioned crores, early retirement, and building "another Anthropic." I love the ambition — hold on to it. But for the next 26 weeks, the ambition is the fuel, not the plan. The plan is boringly specific: master four subjects, ship four projects, rehearse fifteen stories, and run a disciplined application campaign into the UAE, India, and Amsterdam. Wealth follows level; level follows demonstrated capability; capability follows deliberate practice. Do the weeks.

— Your mentor

The Learning Philosophy of This Book

You told me you learn intuition-first, then deep-dive. Good — that is exactly how this book is written. Every chapter follows the same five-layer loop. Do not skip layers.

Layer	What it means	How it shows up in each chapter
1. Intuition	A mental model or analogy you can explain to a PM in 60 seconds	The "Why this exists" and "The intuition" sections
2. Theory	The math and the assumptions — where the method breaks	"The curriculum" sections and the primary readings
3. Code	Implement the core idea from scratch, small and ugly, before using a library	The "Build it" exercises
4. Production	Scale, latency, cost, monitoring, failure modes	The "Production lens" notes and system-design prompts
5. Teaching	Write or say it out loud — a blog post, a mock interview, a whiteboard session	The "Checkpoint" questions; if you cannot teach it, you do not own it

The Feynman rule for this entire program: every Sunday evening, pick one concept you studied that week and explain it out loud, on a whiteboard, in under five minutes, with one diagram and one equation. Record yourself.

This single habit is worth more than ten hours of passive video watching, and it doubles as interview practice.

How To Use This Book

- **Chapters are ordered by dependency, not by importance.** Transformers (Ch. 6) come before LLMs and fine-tuning (Ch. 8) because you cannot understand LoRA without understanding attention weight matrices. Experimentation (Ch. 1) comes first because it takes the longest to mature and it is the #1 filter at your top-target companies.
- **Three parallel tracks run every single week:** Learning (5–6 hrs), Building (6–8 hrs), Interview practice (3–4 hrs). The week-by-week calendar in Part V weaves them together; the chapters give you the depth.
- **Resources are ranked.** Within each chapter, the first resource listed is the primary text — do it fully. Others are supplements: use them when the primary explanation does not click.
- **Checkpoints are pass/fail.** At the end of each chapter is a list of "Can you...?" questions. If the answer to any is no, you are not done with the chapter, no matter how many videos you watched.
- **Everything feeds the resume.** Each chapter ends with the interview and resume signal it unlocks. By Week 19 your resume should be rewritten almost entirely from artifacts produced by this program.

Your Starting Position — The Honest Snapshot

Area	Today	Target	The gap in one line
Experimentation and A/B testing	5/10	9/10	You have run tests; you cannot yet design them under network effects, ratio metrics, or sequential peeking
Causal inference	3/10	8/10	Near-zero footprint; this is a hard requirement at Booking, Uber, Airbnb
Modern RecSys and ranking	6/10	9/10	Deep practical experience trapped in 2016-era vocabulary (Wide and Deep, pointwise CTR)
Deep learning and PyTorch	4/10	7/10	You have used DL products but cannot yet write a training loop from scratch fluently
GenAI: Transformers, RAG, fine-tuning, agents	3/10	8/10	One strong GPT-4o bullet; no architectural depth, no evaluation rigor, no agents
SQL and coding under pressure	5/10	8/10	Daily-driver SQL, but not sessionization/funnels at interview speed
Staff-level leadership evidence	4/10	8/10	You have influenced cross-functionally but never packaged mentoring, direction-setting, or org impact

PART I

Foundations That Compound — Weeks 1–8|

Chapter 1 — Experimentation and A/B Testing at Scale

Weeks 1–2 · Priority P1 · The single highest-ROI subject in this entire book

1.1 Why This Is Chapter One

At Booking.com, the interview is essentially an experimentation exam with a company attached. Uber, Airbnb, and Google product-DS loops all devote at least one full round to it. And unlike LeetCode, experimentation knowledge compounds: it powers your causal inference chapter, your product-sense answers, your portfolio Project 4, and every "how did you measure that?" follow-up on every resume bullet you have. You wrote "achieving 4.6% CTR lift" — the very first Staff-level follow-up will be: what was the experimental design, the unit of randomization, the power, and the guardrails? Right now you cannot answer at depth. In two weeks, you will.

1.2 The Intuition

An A/B test is a machine for manufacturing counterfactuals. You cannot observe the same user in two universes, so you build two statistically identical populations and let each live in one universe. Every advanced topic in this chapter is a repair job for when that machine breaks: users contaminate each other (network effects), the metric is a ratio of two random quantities (delta method), you peek at results early (sequential testing), you run 40 tests and 2 "win" by luck (multiple testing), or your randomization unit differs from your analysis unit (clustered designs). Learn the ideal machine first, then learn every way it breaks — interviews live almost entirely in the breakage.

1.3 The Curriculum

Week 1 — The core machine

- Hypothesis testing from first principles: null/alternative, Type I and II errors, p-values (what they are and are not), confidence intervals, and why $\alpha = 0.05$ is a convention, not a law.
- Power analysis and sample size: the four-way trade-off between effect size (MDE), power, α , and n . Derive the standard sample-size formula once by hand; then build a calculator.
- Metric design: success metrics vs guardrail metrics vs debugging metrics; sensitivity vs directionality; the Overall Evaluation Criterion (OEC) concept from Kohavi.
- Randomization units: user vs session vs page-view vs cluster, and why the choice silently determines your variance and your independence assumptions.
- Practicalities that interviews love: A/A tests, sample-ratio mismatch (SRM) as the #1 data-quality alarm, novelty and primacy effects, and pre-experiment bias checks (CUPED as a teaser).

Week 2 — Where the machine breaks

- Ratio metrics and the delta method: $\text{CTR} = \text{clicks}/\text{views}$ is a ratio of two correlated random variables; naive variance is wrong. Understand the delta method fix and when bootstrap is the pragmatic alternative.
- Variance reduction: CUPED (using pre-experiment data as a covariate) — how Booking and Microsoft cut experiment duration by 30–50%.

- Sequential testing and peeking: why checking daily inflates false positives; always-valid p-values, mSPRT, and group-sequential designs as fixes.
- Multiple testing: Bonferroni vs Benjamini–Hochberg (FDR); when each is appropriate.
- Interference and network effects: two-sided marketplaces (Careem!) where treating drivers changes the rider experience; cluster-randomized and switchback experiments as answers.
- Heterogeneous effects and segment analysis: why post-hoc segment fishing is dangerous, and how to pre-register cuts.
- When you cannot randomize at all: the bridge into Chapter 2.

1.4 Resources — Ranked

Resource	Type	How to use it
Kohavi, Tang and Xu — "Trustworthy Online Controlled Experiments" (Cambridge, 2020)	Book (primary)	THE text. Read Ch. 1–4 in Week 1, Ch. 15–23 (statistics, variance, pitfalls) in Week 2. Annotate; you will re-read before Booking/Uber interviews
exp-platform.com (Kohavi's papers page)	Papers	"Seven Rules of Thumb for Web Site Experimenters" and the SRM paper — short, dense, interview gold
Ron Kohavi's LinkedIn posts and "Statistics for Experimenters" threads	Ongoing	He posts real pitfalls monthly; follow now
Spotify, Netflix, Airbnb and Booking engineering blogs (experimentation tags)	Blogs	One post per day with breakfast; note each company's house methodology (Netflix: quasi-experiments; Booking: peeking culture)
Evan Miller's online tools + "How Not To Run an A/B Test"	Tool + essay	Classic peeking explanation; sanity-check your own calculator against his
Udacity A/B Testing course (by Google, free)	Course (backup)	Only if the book feels dry; it is lighter but well structured

1.5 Build It

- **Build 1 (Week 1):** a sample-size and power calculator in Python — inputs: baseline rate, MDE, alpha, power; outputs: n per arm, expected duration given traffic. Add a CLI and unit tests. This becomes a small public repo and a talking point.
- **Build 2 (Week 1):** take a public A/B test dataset (e.g., the "Cookie Cats" mobile game dataset on Kaggle) and produce a full analysis notebook: SRM check, primary metric with CI, guardrails, segment cuts with FDR correction, and a one-paragraph executive recommendation.
- **Build 3 (Week 2):** simulation study — simulate a clustered experiment (users within cities) and show, with charts, how ignoring clustering understates variance and inflates false positives. Then implement the delta method for a ratio metric and validate against bootstrap.
- **Build 4 (Week 2):** implement mSPRT or an always-valid p-value procedure on simulated streaming data; plot how naive daily peeking produces ~30% false positive rates vs ~5% for the corrected method.

Production lens: for every technique ask — how would this run at Careem scale? Where do assignments get logged? How do you detect SRM automatically? What is the experiment review process before shipping? These operational answers are what separate a Senior answer from a Staff answer.

1.6 Checkpoint — Can You...?

-
- Derive the sample size formula for a two-proportion z-test and explain every term to a PM?
 - Explain why CTR needs the delta method, on a whiteboard, in three minutes?
 - Design an experiment for a Careem driver-incentive change, identifying the interference problem and proposing a switchback or geo-cluster design?
 - Diagnose a sample-ratio mismatch and list five root causes?
 - Explain CUPED in one sentence and state what it does to variance and to bias?
 - Say precisely what a p-value is — and give two things it is not?

Resume signal unlocked: you can now rewrite the Albertsons bullet with "on an A/B test of N users, powered to detect X%, with position-bias-aware evaluation" — and defend every word of it.

Chapter 2 — Causal Inference

Weeks 3–4 · Priority P1 · Your single biggest zero-to-competent jump

2.1 Why This Exists

A/B tests answer "did it work?" when you can randomize. Most of the highest-stakes business questions arrive when you cannot: What was the true incremental effect of our marketing campaign? Did the price change cause the churn? What would revenue have been without the outage? Airbnb, Uber, and Booking test causal inference explicitly at Staff level, and your Mastercard work in ads and marketing services makes this your most natural differentiator — attribution and incrementality ARE causal inference. Today this is your weakest area (3/10). In four weeks it becomes a strength you can hang a portfolio project on (Project 4).

2.2 The Intuition

All of causal inference is one question: **what is the credible counterfactual?** Randomization manufactures counterfactuals; every observational method is a different strategy for borrowing one. Matching says "find a statistical twin who wasn't treated." Difference-in-differences says "the untreated group's trend tells me what the treated group's trend would have been." Synthetic control says "no single twin exists, so I'll build a Frankenstein twin from a weighted combination of donors." Instrumental variables says "find a lever that moves treatment but touches the outcome only through treatment." Every method is a set of assumptions in exchange for a counterfactual — and interviews are entirely about whether you can name the assumptions and describe how they fail.

2.3 The Curriculum

Week 3 — The framework and the workhorses

- Potential outcomes (Rubin causal model): $Y(1)$, $Y(0)$, the fundamental problem of causal inference, ATE vs ATT vs CATE.
- Confounding, colliders, and DAGs: draw causal graphs; understand the backdoor criterion informally; know why conditioning on a collider creates bias (interviewers love this trap).
- Selection on observables: propensity scores, matching, inverse probability weighting (IPW), and doubly-robust estimators; overlap/positivity diagnostics.
- Regression adjustment and its limits; omitted variable bias in plain language.
- The Lalonde dataset story: why observational estimates of the job-training program went wrong and what fixed them — the canonical case study.

Week 4 — The quasi-experimental toolkit and heterogeneity

- Difference-in-differences: parallel trends, event-study plots, staggered adoption pitfalls (modern DiD literature in one honest paragraph — know it exists).
- Synthetic control: the Abadie California Prop 99 smoking study; donor pools, pre-treatment fit, placebo tests. You will replicate this.
- Instrumental variables: relevance and exclusion restriction; encouragement designs in tech (e.g., randomized promotion of a feature as an instrument for usage).

- Regression discontinuity: sharp vs fuzzy; bandwidth choice; why "users just above/below a threshold" is a gift when you find it.
- Uplift modeling and heterogeneous treatment effects: S/T/X/R meta-learners, causal forests (EconML), and Qini curves — this is the bridge from causal inference to marketing science and directly relevant at Mastercard today.
- Geo-experiments and marketing incrementality: matched-market designs, MMM in one honest overview, incrementality vs attribution — your Project 4 backbone.

2.4 Resources — Ranked

Resource	Type	How to use it
Matheus Facure — "Causal Inference for the Brave and True" (free online)	Book (primary)	Python-first, intuition-first, written by a tech DS. Do Part I fully in Week 3, Part II selectively in Week 4. Run every notebook
Scott Cunningham — "Causal Inference: The Mixtape" (free online)	Book	Deeper on DiD, IV, RDD with real research stories; read the chapters matching each Week 4 topic
Brady Neal — "Introduction to Causal Inference" (free course + book)	Course	Best formal treatment of DAGs and the backdoor criterion; watch Lectures 1–4
EconML (Microsoft) and DoWhy documentation + tutorials	Library	Your tooling for meta-learners and causal forests; work through the customer-scenario notebooks
CausalML (Uber) documentation	Library	Uplift modeling and Qini curves — note that Uber built this; mention it in Uber interviews
Abadie, Diamond, Hainmueller — synthetic control papers (California Prop 99)	Paper	Read once for the method, then replicate (Build 2)
Emily Oster / event-study explainers, and "The Effect" by Nick Huntington-Klein (free)	Backup	When parallel trends or fixed effects will not click

2.5 Build It

- **Build 1 (Week 3):** Lalonde dataset — estimate the training program effect with naive comparison, regression, PSM, and IPW; compare all against the experimental benchmark in a single table and write 200 words on why they diverge.
- **Build 2 (Week 4):** replicate the California Prop 99 synthetic control study end-to-end, including placebo tests across donor states and a permutation-based p-value.
- **Build 3 (Week 4):** on a simulated marketing dataset, run all four meta-learners (S/T/X/R) plus a causal forest via EconML; compare CATE estimates and plot Qini curves; write up which you would deploy for a Mastercard-style campaign-targeting problem and why.
- **Stretch:** a difference-in-differences event-study plot on any public policy dataset with staggered adoption, and one paragraph on why staggered DiD is subtle.

Career note: everything in Week 4 is Project 4 in embryo. Do these builds properly and the portfolio project becomes assembly rather than invention.

2.6 Checkpoint — Can You...?

- State the fundamental problem of causal inference in one sentence?

-
- Draw a DAG with a confounder and a collider and explain which one to condition on and why?
 - List the identifying assumption of PSM, DiD, synthetic control, IV, and RDD — one line each — and give a realistic way each fails in a tech setting?
 - Explain to a marketing VP why last-click attribution is not incrementality, using a concrete example?
 - Choose between DiD and synthetic control for "we launched in Dubai only" and defend the choice?
 - Explain what a causal forest estimates that XGBoost-on-outcomes does not?

Chapter 3 — PyTorch and Deep Learning Foundations

Weeks 5–8, parallel track · Priority P1 · The substrate every later chapter runs on

3.1 Why This Exists

You cannot fine-tune an LLM, implement a Two-Tower model, or discuss attention at a whiteboard without fluent PyTorch. Your resume lists no PyTorch, and "no hands-on DL framework" is an instant down-level at Nvidia, Google, Amazon, and Databricks. The fix is not a survey course — it is writing training loops with your own hands until tensors, autograd, and gradient descent are muscle memory. This chapter runs as a parallel track under Chapters 4–7: every model you learn there, you implement here.

3.2 The Intuition

A neural network is a differentiable program: a chain of matrix multiplications and nonlinearities whose every knob can be nudged in the direction that reduces error, because calculus gives us the direction (the gradient) and autograd computes it mechanically via the chain rule. Once you internalize "forward pass = compute, backward pass = assign blame proportionally," every architecture — CNN, LSTM, Transformer — is just a different wiring diagram over the same three primitives: linear maps, nonlinearities, and loss functions.

3.3 The Curriculum

- Tensors, broadcasting, autograd mechanics; write backprop by hand once for a 2-layer MLP (matrix calculus refresh — Nvidia and Google still ask this).
- The canonical training loop: dataset/dataloader, forward, loss, backward, optimizer step, eval loop, early stopping, checkpointing.
- Optimization: SGD vs momentum vs Adam/AdamW; learning-rate schedules and warmup; why weight decay is not quite L2 in Adam.
- Regularization and stability: dropout, batch/layer norm (and why Transformers use LayerNorm), gradient clipping, mixed precision (fp16/bf16) at a working level.
- Embeddings: nn.Embedding as a learned lookup table — the single most important concept for RecSys and NLP alike.
- Debugging DL: overfit a single batch first; loss-curve pathology reading; the "it trains but does not learn" checklist.

3.4 Resources — Ranked

Resource	Type	How to use it
Andrej Karpathy — "Neural Networks: Zero to Hero" (YouTube, free)	Course (primary)	The best DL pedagogy on the internet. Do micrograd and makemore parts 1–3 in Weeks 5–6, coding along, pausing constantly. The GPT video belongs to Chapter 6
"Dive into Deep Learning" (d2l.ai, free)	Book	Reference for anything Karpathy moves fast on: norms, optimizers, attention math. PyTorch tab

Resource	Type	How to use it
Official PyTorch tutorials ("Learn the Basics", "What is torch.autograd")	Docs	Weekend 1 of Week 5; do not skip the autograd one
3Blue1Brown — Neural networks series (YouTube)	Visual	One evening; the backprop visualizations will lock in the intuition permanently
Sebastian Raschka — "Machine Learning with PyTorch and Scikit-Learn"	Book (backup)	If you prefer a linear textbook path

3.5 Build It

- Micrograd from scratch (following Karpathy, then re-doing key parts without the video).
- A 2-layer MLP on MNIST or Fashion-MNIST with a hand-rolled training loop — no Lightning, no shortcuts — reaching a respectable accuracy, with a proper train/val/test protocol and a loss-curve writeup.
- The Chapter 4 and 6 builds (Two-Tower, mini-Transformer) are the real graduation exams of this chapter.

3.6 Checkpoint — Can You...?

- Write a full PyTorch training loop from a blank file, no references, in under 20 minutes?
- Derive the gradient of a softmax-cross-entropy layer on paper?
- Explain what autograd stores during the forward pass and releases after backward?
- Diagnose from a loss curve: too-high LR, no shuffling, data leakage, and dead ReLUs?

Chapter 4 — Modern Recommender Systems

Week 5 (learning) + ongoing · Priority P1 · Upgrading your strongest domain to 2026 vocabulary

4.1 Why This Exists

Personalization is your deepest domain — Albertsons ranking, Innovaccer VAE recommender, Myntra demand — and the single most in-demand DS specialty at your target companies (Careem, Talabat, Booking, Zalando, LinkedIn). But your vocabulary is dated: Wide & Deep (2016) reads like "I stopped learning in 2021." The gap is not conceptual; it is architectural literacy. One focused week plus ongoing practice converts eight years of recsys experience into fluent 2026 language, and it will feel like renaming things you already know.

4.2 The Intuition

Every industrial recommender is a funnel with two brains. **Candidate generation** (retrieval) answers "from 100 million items, which 500 might this user care about?" — it must be fast, so we use embeddings and approximate nearest-neighbor search: represent users and items as vectors such that dot-product = affinity. **Ranking** answers "of these 500, in what order?" — it can be slow and feature-rich, so we use heavy models with cross-features. The Two-Tower model IS candidate generation (user tower, item tower, dot product, trained with in-batch negatives). Your Wide & Deep was a ranking-stage model; DeepFM, DCN, and DIN are its modern descendants that learn feature interactions automatically instead of via manual crosses. Multi-task heads (MMoE/PLE) exist because businesses care about clicks AND conversions AND dwell — and those objectives fight each other.

4.3 The Curriculum

- The two-stage (often four-stage: retrieval → pre-rank → rank → re-rank) architecture; where diversity, freshness, and business rules get injected.
- Candidate generation: matrix factorization → Two-Tower with in-batch softmax and negative-sampling subtleties (why in-batch negatives bias against popular items, and logQ correction); ANN search — FAISS/ScaNN, IVF and HNSW at intuition level.
- Ranking architectures lineage: Wide & Deep → DeepFM → DCN-v2 → DIN/DIEN (attention over user behavior sequences) → multi-task MMoE/PLE. For each: the one problem it solves over its predecessor.
- Cold start: content features, item-to-item fallbacks (you built this at Albertsons — now name it properly), and bandits/exploration (epsilon-greedy → Thompson sampling at working level).
- Evaluation: offline (recall@k, NDCG, MRR, calibration) vs online (CTR, conversion, dwell, diversity indices); why offline-online correlation breaks; interleaving as a cheap online signal.
- Feedback loops and position bias — the bridge to Chapter 5.
- Sequence models in modern recsys: SASRec/BERT4Rec at reading level (transformers eat recsys too — connects to Ch. 6).

4.4 Resources — Ranked

Resource	Type	How to use it
Eugene Yan's blog — recsys system-design posts ("System Design for Recommendations and Search", "Patterns for Personalization")	Blog (primary)	The best industrial recsys writing available; read the entire recsys tag, take architecture notes
Covington et al. — "Deep Neural Networks for YouTube Recommendations" (2016)	Paper	THE canonical two-stage paper; read twice; every recsys interview references its ideas
Papers: DeepFM (2017), DCN-v2 (2020), DIN (2018), MMoE (2018)	Papers	One per evening; read for the core idea and Figure 1, not every equation
Google's Machine Learning "Recommendation Systems" course (free)	Course	Fast structured pass on retrieval/ranking/re-ranking if you want scaffolding
Meta DLRM paper + blog	Paper	Industrial-scale ranking; good for scale talking points
Gaurav Chakravorty / recsML and "System Design for ML" interview repos on GitHub	Prep	Mine for interview-style recsys design questions

4.5 Build It

- **Two-Tower on MovieLens-1M in PyTorch (the flagship build):** user tower + item tower, in-batch softmax negatives, recall@k evaluation against a matrix-factorization baseline, then FAISS index for retrieval and a latency benchmark. Document the logQ popularity correction even if simplified.
- Fine-tune the evaluation: produce a table of recall@10/50/100 and NDCG for MF vs Two-Tower, and a 300-word README section explaining the delta.
- Reading exercise: rewrite your Albertsons and Innovaccer resume bullets using modern vocabulary — you will be shocked how much stronger they read with zero exaggeration.

4.6 Checkpoint — Can You...?

- Whiteboard the full four-stage architecture for "recommend restaurants on Talabat" in 10 minutes, including where cold-start, diversity, and business constraints live?
- Explain why in-batch negatives are biased and how logQ correction fixes it?
- Compare DeepFM vs DCN vs DIN in three sentences — what each adds?
- Explain when offline NDCG improvements fail to show up in online A/B tests, and name three causes?
- Defend your Innovaccer VAE choice against "why not matrix factorization or Two-Tower?" — with a real trade-off answer?

Chapter 5 — Learning to Rank

Week 6 · Priority P1 · The sister discipline that makes your ranking claims defensible

5.1 Why This Exists

Search and feed ranking interviews (Google, Booking, Uber, Zalando fashion search) assume fluency in learning-to-rank: pointwise vs pairwise vs listwise, LambdaMART, and position bias. You have shipped ranking without this vocabulary. One week closes it, because LTR is a small, well-bounded field with one dominant algorithm family and one dominant pathology.

5.2 The Intuition

Pointwise ranking ("predict CTR per item, sort by score") optimizes the wrong thing: you are graded on the ORDER, not on per-item accuracy. Pairwise methods (RankNet) fix this by learning from preferences — "item A should beat item B" — and listwise refinements (LambdaRank/LambdaMART) weight each pairwise mistake by how much it damages NDCG, so mistakes at rank 1 hurt more than mistakes at rank 50. The dominant pathology is **position bias**: users click position 1 because it is position 1, so naive training data teaches the model "whatever we ranked first is best" — a self-fulfilling feedback loop. Inverse propensity scoring (IPS) reweights clicks by how visible the position was, breaking the loop.

5.3 The Curriculum

- Ranking metrics deeply: NDCG (derive DCG by hand once), MRR, MAP; why NDCG@k is the industry default.
- Pointwise → pairwise (RankNet) → listwise (LambdaRank, LambdaMART): Burges's unified story; why LambdaMART (gradient-boosted trees + lambda gradients) still wins most tabular ranking benchmarks in 2026.
- Position bias and unbiased LTR: examination hypothesis, IPS, and result randomization vs intervention-free propensity estimation; joint learning of relevance and propensity.
- Multi-objective ranking: blending relevance, diversity (MMR), freshness, and business value; cascading rankers.
- Real-time vs batch features in ranking; feature freshness as a ranking lever.

5.4 Resources — Ranked

Resource	Type	How to use it
Chris Burges — "From RankNet to LambdaRank to LambdaMART: An Overview" (Microsoft TR)	Paper (primary)	The whole field in 20 pages, by its inventor; read twice, derive the RankNet gradient once
LightGBM / XGBoost ranking documentation (lambdarank objective)	Docs	Your practical tool; note the label-gain and group-structure details
Thorsten Joachims — unbiased learning-to-rank papers and lecture videos	Papers	The position-bias canon; the SIGIR tutorial slides are the fastest entry

Resource	Type	How to use it
Airbnb "Applying Deep Learning to Search Ranking" + Amazon/Etsy search blogs	Blogs	Industrial war stories; excellent interview anecdotes

5.5 Build It

- LambdaMART via LightGBM on the MSLR-WEB10K or Yahoo LTR dataset: proper group-aware splits, NDCG@10 reporting, feature-importance analysis.
- Position-bias simulation: generate synthetic click logs with a known examination model, show that naive training locks in the bias, then implement IPS-weighted training and show recovery. This makes a superb short blog post.
- Bonus: a small pairwise RankNet in PyTorch (ties Chapters 3 and 5 together).

5.6 Checkpoint — Can You...?

- Explain pointwise vs pairwise vs listwise with the "graded on order" framing in two minutes?
- Write the NDCG formula from memory and explain the log discount?
- Describe the position-bias feedback loop and two mitigation strategies, one requiring randomization and one not?
- Design the ranking layer for Booking.com hotel search, including multi-objective blending and how you would evaluate it online?

Chapter 6 — Transformers From First Principles

Week 7 · Priority P1 · The gateway: nothing in Part II makes sense without this

6.1 Why This Exists

You asked for a book where "learning GPT requires learning Transformers first" — this is that dependency made explicit. Every subject in Part II (LLMs, RAG embeddings, LoRA, agents) is an application of one architecture. Interviews at Microsoft, Google, Walmart, and Talabat will probe whether your GenAI knowledge is API-deep or architecture-deep; the difference is exactly one honest week with attention math and one from-scratch implementation.

6.2 The Intuition

A Transformer is a machine for letting every token ask questions of every other token. Each token emits a **query** ("what am I looking for?"), a **key** ("what do I contain?"), and a **value** ("what will I contribute if you attend to me?"). Attention scores = queries dotted with keys (scaled, softmaxed) = "how relevant is every other token to me?"; the output = attention-weighted sum of values = "a new representation of me, informed by everything relevant." Stack this with feed-forward layers, residual connections, and LayerNorm, and you get a architecture that is parallelizable (unlike your old LSTMs — this is WHY Transformers won), scalable, and shockingly general. GPT is just this, decoder-only, trained to predict the next token, at obscene scale.

6.3 The Curriculum

- Tokenization: BPE intuition, vocabulary trade-offs, why tokenizers cause weird LLM failures (counting letters, arithmetic).
- Embeddings + positional information: learned positions → RoPE (reading level).
- Self-attention step by step: Q/K/V, scaled dot-product, the softmax, multi-head as "multiple conversation channels"; causal masking for decoders.
- The block: attention → FFN, residual streams, pre-LayerNorm; parameter counting (be able to estimate why a model is "7B").
- Encoder vs decoder vs encoder-decoder: BERT vs GPT vs T5 — and which powers embeddings (Ch. 7) vs generation (Ch. 8).
- Training vs inference: teacher forcing, next-token loss; KV-cache and why long contexts are expensive; sampling — temperature, top-p, greedy vs beam.
- Scaling picture in one page: pretraining → instruction tuning → RLHF/DPO, so Chapter 8 lands on prepared ground.

6.4 Resources — Ranked

Resource	Type	How to use it
Karpathy — "Let's build GPT: from scratch, in code" + nanoGPT repo	Video/code (primary)	The centerpiece of the week. Code along fully; then rebuild the attention block from a blank file
Jay Alammar — "The Illustrated Transformer" and "The Illustrated GPT-2"	Blog	Read BEFORE Karpathy; the visuals give you the map

Resource	Type	How to use it
Vaswani et al. — "Attention Is All You Need" (2017)	Paper	Read AFTER building; it will finally be easy
3Blue1Brown — "Attention in transformers, visually explained"	Video	One evening; cements the geometry of Q/K/V
Hugging Face NLP Course (free), Ch. 1–3	Course	Bridges from-scratch understanding to the practical HF ecosystem you use in Part II
Sebastian Raschka — "Build a Large Language Model (From Scratch)"	Book (backup)	If you want a slower, complete textbook path; also serves Chapter 8

6.5 Build It

- nanoGPT-style character-level Transformer trained on a small corpus (Shakespeare or your own Medium posts — the latter makes a fun demo), with your own attention implementation.
- Attention visualization notebook: heatmaps of attention weights on example sentences; write 300 words interpreting two heads.
- Parameter-count exercise: compute, by hand, the parameter budget of a 12-layer, 768-dim, 12-head model and check against GPT-2 small.

6.6 Checkpoint — Can You...?

- Write scaled dot-product attention in PyTorch from memory, including the causal mask?
- Explain why Transformers replaced LSTMs, using the words "parallelism" and "long-range dependencies"?
- Explain what the KV-cache stores and why inference cost grows with context length?
- Trace one token's journey through a decoder block on a whiteboard?
- Explain temperature and top-p to a PM deciding chatbot settings?

Chapter 7 — RAG Fundamentals

Week 8 · Priority P1 · Renaming and upgrading the system you already built at Innovaccer

7.1 Why This Exists

Here is a secret weapon hiding in your resume: your Innovaccer GPT-4o medical-guidelines system IS a RAG system — you built one before you knew the acronym. This chapter gives you the architecture vocabulary, the design space, and the evaluation rigor to retell that story at Staff level, and it is the foundation of portfolio Project 2. In 2026, RAG design is the single most common GenAI system-design interview question.

7.2 The Intuition

LLMs have two memory problems: their knowledge is frozen at training time, and they confabulate when asked about things they do not know. RAG bolts on an open-book exam: instead of asking the model to remember, you retrieve the relevant pages and paste them into the prompt. The entire discipline is then three questions: (1) how do you cut the library into retrievable pieces (chunking), (2) how do you find the right pieces fast (embeddings + vector search + reranking), and (3) how do you know the system is telling the truth (evaluation: retrieval quality AND generation faithfulness, separately). Notice that question 2 is literally your Chapter 4 retrieval stack — embeddings and ANN search — wearing a text costume. Your recsys expertise transfers almost one-to-one.

7.3 The Curriculum

- Chunking: fixed vs recursive vs semantic vs document-structure-aware; chunk size vs retrieval granularity trade-off; metadata and headers as retrieval features.
- Embedding models: sentence embeddings (SBERT lineage), the MTEB leaderboard as your shopping guide, bge/gte/e5 families; when to fine-tune embeddings.
- Vector databases: FAISS vs Chroma vs Qdrant vs Pinecone — index types (flat, IVF, HNSW), the recall/latency/cost triangle, and the honest answer that pgvector is fine for most startups.
- Hybrid retrieval: dense + BM25, reciprocal rank fusion; rerankers (cross-encoders) and why bi-encoder retrieval + cross-encoder rerank is the standard cascade — the same retrieval-then-rank pattern as Chapter 4.
- Query understanding: rewriting, decomposition, HyDE at reading level.
- Evaluation: retrieval metrics (recall@k, MRR, NDCG — old friends), generation metrics (faithfulness, answer relevance), RAGAS framework, LLM-as-judge with its biases, and building a golden test set.
- Failure modes: lost-in-the-middle, context stuffing vs precision, stale indexes, chunk-boundary answers.

7.4 Resources — Ranked

Resource	Type	How to use it
DeepLearning.AI short courses: "Building and Evaluating Advanced RAG" and "Advanced Retrieval for AI with Chroma" (free)	Courses (primary)	Two focused afternoons; Andrew Ng's short-course format matches your intuition-first style

Resource	Type	How to use it
Pinecone Learning Center (RAG, vector-index, and rerankers series)	Guides	The best written explanations of index internals and chunking trade-offs
RAGAS documentation and paper	Framework	Your evaluation backbone for Project 2; run the quickstart this week
LlamaIndex and LangChain RAG documentation	Docs	Learn LlamaIndex concepts even if you build with LangChain — its retrieval abstractions teach the design space
Anthropic's "Contextual Retrieval" and similar engineering posts	Blog	A modern chunking-enrichment technique worth name-dropping with understanding
Eugene Yan — "Patterns for Building LLM Systems and Products"	Essay	The industrial map of the whole LLM-systems space; re-read monthly through Part II

7.5 Build It

- A small but complete RAG system over a document set you genuinely know (e.g., the Kohavi book notes you made in Week 1, or clinical-guideline PDFs): recursive chunking, bge embeddings, Chroma, hybrid BM25+dense with RRF, cross-encoder rerank, GPT-4o or Llama generation.
- A golden set of 30 Q&A pairs; evaluate with RAGAS (faithfulness, answer relevance, context precision/recall); compare 2 chunking strategies and with/without reranking in a results table.
- Rewrite the Innovaccer GenAI resume bullet using this vocabulary — honestly, describing what you actually built, now properly named.

7.6 Checkpoint — Can You...?

- Whiteboard a production RAG architecture end-to-end in 10 minutes, with an evaluation plan?
- Explain the bi-encoder vs cross-encoder trade-off and why the cascade exists?
- Name three chunking strategies and when each wins?
- Explain how you would detect and reduce hallucination in a clinical RAG system — and why "temperature 0" is not an answer?
- Argue when RAG beats fine-tuning and vice versa (preview of Chapter 8)?

PART II

The GenAI Stack — Weeks 9–18|Learned in parallel with the portfolio projects of Part III

Chapter 8 — LLM Internals and Fine-Tuning (LoRA / QLoRA)

Weeks 9–10 · Priority P1 · Where API users get separated from applied scientists

8.1 Why This Exists

Anyone can call an API. Staff Applied Scientists are hired to know when and how to adapt models: full fine-tuning vs parameter-efficient methods, dataset curation, and evaluation of the result. LoRA questions appear in roughly 40% of 2026 GenAI loops, and Project 3 (LLM-generated recommendation explanations) is built entirely on this chapter. This is also where your Chapter 6 investment pays out: LoRA is incomprehensible as an API flag but obvious once you know attention weight matrices.

8.2 The Intuition

Fine-tuning a 7B-parameter model means updating 7 billion numbers — expensive in GPU memory (gradients and optimizer states triple the bill) and risky (catastrophic forgetting). LoRA's insight: the CHANGE needed to adapt a model is low-rank — you do not need to repaint the whole house, you need a thin stencil layered on top. So freeze the original weight matrix W and learn a correction as the product of two skinny matrices $A \cdot B$ (rank 8 or 16 instead of 4096). You train perhaps 0.5% of the parameters, get most of the quality, and can hot-swap adapters per task. QLoRA goes further: store the frozen base in 4-bit precision so the whole thing fits on one consumer GPU. That is the entire idea — everything else is engineering detail.

8.3 The Curriculum

- The adaptation ladder: prompting → few-shot → RAG → PEFT → full fine-tuning → pretraining; the cost/control/data trade-off at each rung, and how to choose (a classic interview question).
- The training stack refresher: causal-LM loss on completions, instruction formats and chat templates, packing, masking the prompt tokens.
- LoRA mechanics: rank, alpha, target modules (why attention projections first), merging adapters; QLoRA — NF4 quantization, double quantization, paged optimizers, at working level.
- Dataset curation: quality over quantity, instruction diversity, contamination checks; why 1,000 excellent examples often beat 100,000 scraped ones (LIMA finding).
- Preference tuning at reading level: RLHF → DPO in one honest page; know what DPO optimizes and why it displaced PPO for most teams.
- Practical training: Hugging Face PEFT + TRL (SFTTrainer), Unsloth for speed on a single GPU, wandb tracking, and evaluation before/after (perplexity is not enough — bridge to Chapter 9).
- Serving fine-tuned models: merging vs adapter-serving, quantized inference (GGUF/AWQ at reading level), vLLM as the default serving engine.

8.4 Resources — Ranked

Resource	Type	How to use it
Hugging Face PEFT documentation + LoRA conceptual guides	Docs (primary)	Read the concepts, then run the official LoRA fine-tuning notebook end-to-end before touching your own data

Resource	Type	How to use it
LoRA paper (Hu et al., 2021) and QLoRA paper (Dettmers et al., 2023)	Papers	Read LoRA fully (it is readable); QLoRA sections 1–3 + skim the rest
DeepLearning.AI — "Finetuning Large Language Models" short course (free)	Course	Half a day; the decision framework (when to fine-tune vs RAG) is interview-ready
Sebastian Raschka — LoRA/finetuning articles and "Build an LLM From Scratch" Ch. on finetuning	Articles	The most lucid technical explainers of rank/alpha choices and experiments
Hugging Face TRL documentation (SFTTrainer, DPOTrainer)	Docs	Your actual training harness for Project 3
Unsloth documentation and Colab notebooks	Tool	Makes Llama-3-8B QLoRA feasible on free/cheap GPUs; use for Project 3

8.5 Build It

- Warm-up: QLoRA fine-tune Llama-3-8B (or Qwen-2.5-7B) on a small public instruction dataset via Unsloth; log loss curves in wandb; compare base vs tuned outputs on 20 held-out prompts.
- The real build is **Project 3** (Chapter 12): fine-tune the model to generate recommendation explanations, with human-preference evaluation. Weeks 9–11.
- Ablation for depth: train rank 4 vs 16 vs 64 on the same data and write 300 words on the quality/parameter trade-off you observed. This tiny experiment is a fantastic interview story — it shows scientist behavior.

8.6 Checkpoint — Can You...?

- Explain LoRA with the low-rank-update intuition AND write the $W + BA$ math?
- Estimate GPU memory for full fine-tuning vs LoRA vs QLoRA of a 7B model and explain where the savings come from?
- Give a crisp decision rule for RAG vs fine-tuning vs both, with a real example of each?
- Describe your dataset curation process and two contamination risks?
- Explain what DPO does differently from RLHF in two sentences?

Chapter 9 — LLM and RAG Evaluation

Weeks 10 and 14 · Priority P1 · The discipline that makes every GenAI claim on your resume defensible

9.1 Why This Exists

The fastest way to spot an API tourist in an interview: ask "how did you know your LLM system worked?" If the answer is "we looked at some outputs," the interview is over. Evaluation is where your classical-ML rigor (AUPRC, A/B tests, calibration) becomes a superpower in GenAI — most GenAI practitioners are weak here and you are structurally strong. Every portfolio project ships with an evaluation report because of this chapter, and that habit alone will distinguish your GitHub from 95% of GenAI portfolios.

9.2 The Intuition

Generation has no single ground truth, so evaluation decomposes into layered proxies. Reference-based metrics (ROUGE, BLEU, BERTScore) ask "does the output resemble a known good answer?" — cheap, but blind to valid paraphrases and useless without references. LLM-as-judge asks a strong model to grade rubric dimensions — scalable and surprisingly aligned with humans, but biased (position bias, verbosity bias, self-preference). Human evaluation is the gold standard — expensive, so you spend it wisely: to calibrate your judge, then let the judge scale. And for RAG specifically, always evaluate retrieval and generation SEPARATELY: a wrong answer from correct context is a generation bug; a wrong answer from wrong context is a retrieval bug. Confusing the two wastes weeks.

9.3 The Curriculum

- Reference-based metrics and their failure modes: ROUGE/BLEU (n-gram overlap), BERTScore (embedding similarity); why they mislead on open-ended generation.
- LLM-as-judge: pairwise vs pointwise grading, rubric design, known biases and mitigations (position swapping, length normalization, separate judge model); calibrating judge scores against a human-labeled subset.
- Human evaluation design: annotation guidelines, inter-annotator agreement (Cohen's kappa), preference tests, sample-size thinking (Chapter 1 pays off again).
- RAG-specific: RAGAS metric family — faithfulness, answer relevance, context precision/recall; building and maintaining a golden dataset; regression testing prompts like code.
- Hallucination: taxonomy (intrinsic vs extrinsic), detection approaches (NLI-based checks, self-consistency, citation verification), and honest limits.
- Online evaluation of GenAI features: implicit signals (regeneration, copy events, thumbs), A/B testing LLM features (Chapter 1 again), and guardrail metrics for safety/latency/cost.
- Benchmarks literacy: MMLU-type benchmarks, MT-Bench/Arena-style preference leaderboards — what they measure and why leaderboard rank does not equal your-task performance.

9.4 Resources — Ranked

Resource	Type	How to use it
Eugene Yan — "Evals" essays (evaluating LLM systems, LLM-as-judge)	Essays (primary)	The best practitioner treatment; read everything tagged evals
RAGAS documentation + paper	Framework	Implement, do not just read; core of Projects 2 and 3
"Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" (Zheng et al., 2023)	Paper	The canonical judge-bias catalogue; skim for the bias sections
Hamel Husain — "Your AI Product Needs Evals"	Essay	The operational playbook: golden sets, error analysis loops, dashboards
DeepLearning.AI — "Evaluating and Debugging Generative AI" (free)	Course	Optional structured pass with wandb tooling

9.5 Build It

- For Project 2: full RAGAS pipeline + a 30-item golden set + a written evaluation report with error analysis (categorize every failure: retrieval vs generation vs question ambiguity).
- For Project 3: human preference evaluation — 3 annotators (recruit friends/colleagues), 100 comparisons, Cohen's kappa, and an LLM-judge calibrated against the human subset; report agreement.
- Write one blog post: "How I evaluated my RAG system like a scientist" — this post alone will outperform most GenAI portfolios.

9.6 Checkpoint — Can You...?

- Design a full evaluation plan for a clinical RAG assistant in 10 minutes, covering offline, judge, human, and online layers?
- Name three LLM-judge biases and a mitigation for each?
- Explain why faithfulness and answer relevance are different failure axes?
- Describe how you would A/B test an LLM feature, including guardrails for latency and cost?

Chapter 10 — Agentic AI Systems

Weeks 11–13 · Priority P1 · The frontier signal for 2026 loops — and Project 1's engine

10.1 Why This Exists

"Agentic AI" is the phrase on every 2026 job description at Walmart, Microsoft, and Talabat, yet most candidates can only recite marketing copy. Understanding what agents actually are — and, more importantly, when they fail and when NOT to use them — positions you above the hype. Project 1 (multi-agent marketing campaign optimizer) is your demonstration piece, chosen deliberately because it fuses agents with your Mastercard marketing-science domain.

10.2 The Intuition

An LLM is a brain in a jar: it can only emit text. An agent is that brain given hands (tools), eyes (retrieval/observation), memory (state), and a loop: think → act → observe → think again. That loop is ReAct, and it is 80% of the field. Everything else is engineering around the loop's failure modes: it wanders (add planning), it forgets (add memory/state), one brain gets confused juggling roles (split into multiple specialized agents with an orchestrator), and it occasionally does something expensive or dangerous (add guardrails, budgets, and human approval gates). The mature engineering position — hold this opinion in interviews — is: **use the dumbest structure that works; a workflow with fixed steps beats a free-roaming agent whenever the task is predictable.**

10.3 The Curriculum

- Tool use / function calling mechanics: schemas, structured outputs, parallel calls, error feedback into the loop.
- ReAct deeply: the paper, the prompt pattern, failure modes (hallucinated tool outputs, loops, giving up).
- Workflows vs agents: prompt chaining, routing, orchestrator-worker, evaluator-optimizer patterns; when autonomy is warranted (read Anthropic's "Building Effective Agents" — it is the field's best design-taste document).
- State and memory: short-term (context), episodic (conversation store), long-term (vector memory); LangGraph's graph-state model — nodes, edges, conditional routing, checkpointing, human-in-the-loop interrupts.
- Multi-agent design: supervisor patterns, role specialization, communication cost; why more agents often means more failure surface, not more intelligence.
- Agent evaluation: task success rate, step efficiency, cost per task, trajectory analysis; building an agent test harness — this is thin literature, which means opinions here sound very senior.
- Safety and cost controls: budgets, timeouts, sandboxing tools, approval gates for irreversible actions.

10.4 Resources — Ranked

Resource	Type	How to use it
Anthropic — "Building Effective Agents" engineering post	Essay (primary)	The design-taste canon: workflow patterns vs agents. Internalize; quote its philosophy in interviews

Resource	Type	How to use it
LangGraph documentation + LangChain Academy "Intro to LangGraph" (free)	Course/docs	Your build framework for Project 1; do the academy course in Week 11
ReAct paper (Yao et al., 2022)	Paper	Short and readable; the founding loop
DeepLearning.AI — "AI Agents in LangGraph" and "Multi AI Agent Systems" short courses (free)	Courses	Fast structured passes; pick one, not both
Lilian Weng — "LLM Powered Autonomous Agents" blog post	Essay	The classic survey of planning/memory/tools; skim for the taxonomy
OpenAI function-calling / structured-outputs guides	Docs	Mechanics reference while building

10.5 Build It

- Week 11 warm-up: a single ReAct agent with 3 tools (calculator, search over a local corpus, Python executor) built once with raw API calls (no framework — feel the loop), then once in LangGraph. The contrast teaches you what the framework actually does.
- Weeks 12–13: **Project 1** — the multi-agent campaign optimizer (Chapter 12 spec): Planner, Audience, Creative, and Optimizer agents over LangGraph with explicit state, budget guardrails, and an evaluation harness comparing agent-designed vs heuristic campaigns in simulation.
- Write a failure-analysis appendix in the repo: the 5 dumbest things your agents did and how you constrained them. Interviewers love this more than the successes.

10.6 Checkpoint — Can You...?

- Whiteboard the ReAct loop and name three failure modes with mitigations?
- Argue when a workflow beats an agent, with an example of each from your own project?
- Explain LangGraph state, conditional edges, and human-in-the-loop checkpoints?
- Design an agent evaluation harness: what metrics, what golden tasks, what cost budget?
- Defend multi-agent decomposition against "why not one agent with more tools?"

Chapter 11 — Production GenAI and ML System Design

Weeks 15–18, woven through projects · Priority P1 · Where Staff-level judgment lives

11.1 Why This Exists

Every Staff loop contains at least one system-design round, and the hire/no-hire line is production thinking: latency, cost, monitoring, failure modes, and iteration loops. You already have real production instincts (your observability platform, your Spark pipelines) — this chapter organizes them into the frameworks interviewers expect and extends them to GenAI serving, which has new physics (GPUs, token economics, KV-caches).

11.2 The Intuition

Classical ML serving is CPU-cheap and stateless; LLM serving is a GPU-economics problem. Cost and latency are functions of tokens: input tokens are cheap-ish and parallel (prefill), output tokens are expensive and serial (decode) — so latency is dominated by output length, and the levers are caching (semantic caches, prompt-prefix/KV caching), batching (continuous batching in vLLM), model right-sizing (route easy queries to small models), and quantization. Meanwhile the classical MLOps loop — versioning, monitoring, drift, retraining — does not disappear; it gets a twin: prompt versioning, eval regression suites, and token-cost dashboards. A Staff answer always covers both loops.

11.3 The Curriculum

- ML system design frameworks: clarify → metrics (business + model) → data → features → model → offline eval → serving → online eval → monitoring → iteration. Practice until this spine is automatic.
- Serving patterns: batch vs online vs streaming inference; feature freshness; feature stores (Feast) online/offline consistency — Project 8-style thinking even if you skip that project.
- GenAI serving: vLLM and continuous batching, KV-cache reuse, semantic caching, model routing/cascades, structured-output enforcement, streaming UX; the cost model (tokens × price, GPU-hour math).
- Reliability: timeouts, retries with idempotency, fallbacks (smaller model, cached answer, graceful degradation), circuit breakers; guardrails for prompt injection and PII (working level).
- Monitoring: data drift vs concept drift; GenAI-specific — eval-set regression runs, judge-scored sampling of production traffic, cost/latency SLO dashboards.
- The deployment safety ladder: shadow → canary → progressive rollout → A/B (Chapter 1 closes the loop).

11.4 Resources — Ranked

Resource	Type	How to use it
Chip Huyen — "Designing Machine Learning Systems"	Book (primary)	Read fully across Weeks 15–17; the closest thing to a Staff system-design textbook
Chip Huyen — "AI Engineering" (2025)	Book	The GenAI-era companion: evaluation, serving, and cost chapters especially

Resource	Type	How to use it
Eugene Yan — "applied-llms.org" ("What We Learned from a Year of Building with LLMs")	Essay	Practitioner consensus on LLM production; dense with quotable judgment
vLLM documentation (concepts: continuous batching, paged attention)	Docs	Read the concept pages even if you never self-host; the vocabulary matters
Evidently AI blog (monitoring, drift) and Made With ML (MLOps course, free)	Guides	Fill specific monitoring/MLOps gaps as needed
"Machine Learning System Design Interview" — Ali Aminian and Alex Xu	Book	Interview-format drills for Part IV; buy now, drill in Weeks 15–22

11.5 Build It

- Wrap Project 2's RAG behind FastAPI with streaming responses, semantic caching, a fallback chain, and a latency/cost benchmark table (p50/p95/p99) in the README.
- Add Prometheus-style metrics or structured logging with a simple dashboard to one project; screenshot it — monitoring evidence is rare in portfolios.
- Weekly drill (Weeks 15–22): one full ML/GenAI system design on a whiteboard, spoken aloud, 35 minutes: rotate through — Talabat restaurant ranking, Careem surge/matching, Booking hotel search, clinical RAG assistant, Mastercard campaign-targeting engine, fraud triage with LLM summaries.

11.6 Checkpoint — Can You...?

- Run the full system-design spine on a novel problem, unprompted, in 35 minutes?
- Estimate the monthly cost of a RAG feature at 1M queries/month and name three levers to cut it 5x?
- Explain continuous batching and why it changed LLM serving economics?
- Design the monitoring plan for both a churn model and an LLM assistant — and articulate what differs?

PART III

The Portfolio — Weeks 9–18|Four projects that do the arguing for you

Chapter 12 — The Portfolio: Strategy and the Four Projects

Weeks 9–18 · Your compensating evidence for the missing MS/PhD

12.1 Why This Exists — And the One Rule

Without an advanced degree, your portfolio and your production track record are the entire case. A recruiter or hiring manager who is unsure about the degree box will look for a reason to say yes; four rigorous, well-documented projects with evaluation reports are that reason. The one rule: **quality over quantity, evidence over demo**. A project with an evaluation report, an ablation, a failure analysis, and an architecture diagram beats five weekend hacks. You are building four projects, not ten — chosen to cover agents, RAG, fine-tuning, and causal inference, and to align with your target companies' domains.

Every repo must ship with: (1) a README that opens with the business problem and quantified result, (2) an architecture diagram, (3) an evaluation report with tables, (4) a failure-analysis section, (5) reproducible setup, and (6) a 3–5 minute demo video or hosted Streamlit app. This standard IS the differentiator.

12.2 The Four Projects at a Glance

#	Project	Weeks	Skills demonstrated	Primary audience
1	Multi-Agent Marketing Campaign Optimizer	11–13	Agentic AI, LangGraph, experimentation, marketing science	Talabat, Careem, Mastercard-adjacent marketing-science roles
2	RAG Customer Intelligence Platform	14–16	RAG architecture, hybrid retrieval, RAGAS evaluation, FastAPI serving	Universal — every GenAI loop; Microsoft, Property Finder
3	LLM Recommendation-Explanation Generator	9–11	LoRA/QLoRA fine-tuning, recsys integration, human evaluation	Booking, LinkedIn, Zalando, any recsys-heavy team
4	Causal Impact Measurement for Campaigns	16–18	Geo-experiments, synthetic control, DiD, uplift, dashboards	Booking, Uber, Airbnb-style product-DS and marketing-science loops

Note the ordering: Project 3 starts first (Week 9) because it rides directly on Chapter 8's fine-tuning week; Project 1 follows the agents chapter; Projects 2 and 4 close the program when your RAG-evaluation and causal skills are mature. Two projects done by end of Month 3, four by end of Month 5 — leaving Weeks 19+ clean for interviews.

12.3 Project 3 — LLM Recommendation-Explanation Generator (Weeks 9–11)

Problem. Recommenders are black boxes; explanations increase trust and CTR. Build a system where a classical recommender produces candidates and a LoRA-fine-tuned LLM generates personalized, grounded explanations ("Because you often reorder oat milk and this brand is on offer...").

Architecture. MovieLens or an e-commerce dataset → LightGBM or your Chapter 4 Two-Tower for candidates → prompt-construction layer with user history features → Llama-3-8B + LoRA (Unsloth/PEFT) fine-tuned on synthetic-but-curated explanation pairs → evaluation: human preference vs a rule-based and a zero-shot baseline, plus a calibrated LLM judge.

Milestones. W9: data + baseline recommender + zero-shot explanation baseline. W10: curate 1–2K training pairs, run QLoRA, rank ablation ($r=4/16/64$). W11: human eval (3 raters, 100 comparisons, kappa), judge calibration, README + demo video, ship.

Resume bullet you will have earned: "Fine-tuned Llama-3-8B with QLoRA to generate grounded recommendation explanations; achieved X% human-preference win rate over zero-shot GPT-4o baseline with a rater-calibrated LLM judge."

12.4 Project 1 — Multi-Agent Marketing Campaign Optimizer (Weeks 11–13)

Problem. Campaign design across segments, channels, and creatives is manual and slow. Build a LangGraph system where a Planner agent proposes strategy, an Audience agent segments a simulated customer base, a Creative agent drafts variants, and an Optimizer agent designs and reads simulated A/B tests to iterate — with budget guardrails and human-approval gates.

Why this project is strategically yours: it fuses agents (frontier signal) with experimentation (Chapter 1) and your live Mastercard marketing domain. In interviews it doubles as a story about your day job's future.

Milestones. W11: simulator (customer base with heterogeneous response functions) + single ReAct warm-up agent. W12: four agents on LangGraph with typed state, budgets, and interrupts. W13: evaluation harness — agent campaigns vs heuristic baseline across 20 simulated runs with CIs (Chapter 1 rigor!), failure-analysis appendix, Streamlit control room, ship.

12.5 Project 2 — RAG Customer Intelligence Platform (Weeks 14–16)

Problem. Support and CX teams cannot query millions of reviews/tickets in natural language. Build a RAG platform over a large public review corpus (e.g., Amazon reviews subset or airline complaints) answering questions like "what do customers complain about most after delivery delays?" with cited sources.

Architecture. Ingestion + structure-aware chunking → bge embeddings + Chroma/Qdrant → hybrid dense+BM25 with RRF → cross-encoder rerank → generation with citations → RAGAS + golden-set evaluation → FastAPI with streaming and semantic cache → Streamlit UI.

Milestones. W14: ingestion, chunking comparison (2 strategies), retrieval-only evaluation (recall@k on golden set). W15: hybrid + rerank, generation with citations, full RAGAS report. W16: FastAPI serving with p95 latency table and cost estimate, error analysis (retrieval vs generation failures), demo video, ship.

12.6 Project 4 — Causal Impact Measurement for Campaigns (Weeks 16–18)

Problem. Last-click attribution misprices marketing. Build a framework that measures true incrementality: a geo-experiment designer (power analysis + matched markets), synthetic-control and DiD estimators, and an uplift-modeling module for targeting — demonstrated on simulated campaign data with known ground truth so you can show estimator accuracy.

Why ground-truth simulation matters: it lets you make the killer claim "my framework recovered the true effect within X% while last-click misattributed by Y%" — a scientific result, not a demo.

Milestones. W16: simulator with known true lift + geo-matching and power module. W17: synthetic control + DiD + placebo tests; meta-learner uplift module with Qini curves. W18: Streamlit dashboard, methodology writeup, README, ship — and polish week for all four repos (diagrams, videos, pinned GitHub profile, LinkedIn featured section).

12.7 GitHub and Distribution Strategy

- One pinned profile README linking all four projects with one-line results; consistent repo template across all four.
- Write 3 blog posts over Weeks 12–18 (Medium, where you already publish): the position-bias simulation (Ch. 5), the RAG evaluation writeup (Ch. 9), and the causal framework methodology. Cross-post to LinkedIn; tag relevant topics. These are your referral-conversation openers.
- Do not chase stars; chase specificity. A hiring manager reading one evaluation report is worth 500 drive-by stars.

PART IV

Interview Mastery and the Campaign — Weeks 19–26|Where preparation converts into offers

Chapter 13 — SQL and Coding Under Pressure

Daily habit from Week 1; intensive Weeks 19–23 · The gate before anyone sees your depth

13.1 Why This Exists

SQL is the first technical gate at Uber, Booking, Atlassian, and Google product-DS loops — and it is a speed test, not a knowledge test. You use SQL daily, which is exactly why it is dangerous: daily-driver SQL and interview SQL (sessionization, funnels, retention cohorts, under a clock, thinking aloud) are different sports. The fix is a small daily habit sustained for six months, not a cram.

13.2 The Curriculum

- Window functions to reflex level: ROW_NUMBER/RANK/DENSE_RANK, LAG/LEAD, running and windowed aggregates, frame clauses.
- The interview canon: sessionization (gap-based), funnel conversion, retention cohorts and triangles, top-N per group, deduplication, consecutive-events problems, self-joins for pairs, pivoting.
- Python for DS interviews: pandas fluency (groupby-agg-merge chains, datetime), plus the core algorithm patterns for AS loops — hash maps, two pointers, sliding window, binary search, BFS/DFS, and light DP. You are not grinding 500 LeetCode; you are mastering ~80 curated problems across these patterns.
- ML coding from scratch (Microsoft/Databricks style): k-means, logistic regression with gradient descent, Naive Bayes, train/test split and metrics without sklearn.

13.3 Resources — Ranked

Resource	Type	How to use it
DataLemur (Nick Singh) SQL questions	Practice (primary)	Real company-style DS SQL; 1/day Weeks 1–18, 2/day Weeks 19–23
LeetCode SQL 50 + top SQL hard list	Practice	The canonical set; finish all 50 by Week 12
StrataScratch	Practice	For company-tagged questions before specific interviews
"Ace the Data Science Interview" — Singh and Huo	Book	SQL + stats + product chapters; drill in Weeks 19–24
NeetCode 150 (filtered to arrays, hashing, two pointers, sliding window, binary search, BFS/DFS)	Practice	~60–80 problems total across the program for Applied Scientist coding rounds

13.4 Checkpoint — Can You...?

- Solve a sessionization or retention-cohort problem in under 15 minutes, talking aloud?
- Write k-means from scratch in 25 minutes, with correct convergence handling?
- Pass a timed mock SQL screen (do at least three on interviewing.io or with peers before Week 22)?

Chapter 14 — ML and AI System Design Interviews

Weeks 15–24 · The round that decides your LEVEL, not just the offer

14.1 Why This Exists

Coding rounds decide whether you get hired; system-design rounds decide the level you get hired AT. Staff offers are won here. Everything from Chapters 4, 5, 7, 10, and 11 converges into a repeatable performance: a calm, structured, trade-off-fluent 40 minutes.

14.2 The Method

Use one spine every time, out loud: **(1) Clarify** the product goal, users, scale, latency, constraints — spend a full five minutes here; rushing this is the #1 failure. **(2) Metrics** — business metric, online model metrics, offline metrics, guardrails. **(3) Data and features** — sources, labels (and label delay/bias!), feature families. **(4) Model** — start simple, justify each escalation; present two alternatives and choose with reasons. **(5) Evaluation** — offline protocol, then online experiment design (Chapter 1 shines). **(6) Serving** — retrieval/ranking split, latency budget, caching, feature freshness. **(7) Monitoring and iteration** — drift, feedback loops, retraining triggers, failure modes. Close with the three biggest risks. Interviewers should feel they are watching a tech-lead run a design review.

14.3 The Drill Set — 12 Designs to Master

- Restaurant/feed ranking (Talabat, Careem Eats) — the full four-stage recsys.
- Hotel search ranking with position bias (Booking) — Chapter 5 on stage.
- Ride ETA / marketplace matching and surge (Careem, Uber) — forecasting + optimization flavor.
- Fraud detection at Mastercard scale — imbalance, cost-sensitive decisions, adversarial drift.
- Churn prediction + intervention targeting — where uplift modeling (Ch. 2) beats churn scores.
- Notification/campaign optimization — multi-objective, fatigue guardrails, experimentation.
- Clinical RAG assistant (your Innovaccer story, formalized) — Chapters 7, 9, 11.
- Customer-support agent with tools — Chapter 10 with guardrails and evaluation harness.
- Semantic product search — embeddings + hybrid retrieval + rerank.
- LLM feature A/B test design — GenAI meets Chapter 1.
- Feed diversity/exploration problem — bandits at working level.
- Demand forecasting for fulfillment (your Myntra story, upgraded to modern methods and reconciliation).

Practice cadence: one per week Weeks 15–18, two per week Weeks 19–22, then full mock loops Weeks 23–24. Record yourself; review for structure, silence-filling, and trade-off language.

14.4 Resources

Resource	Type	How to use it
"Machine Learning System Design Interview" — Aminian and Xu	Book (primary)	One chapter per drill topic; mirror its structure until yours is better

Resource	Type	How to use it
Chip Huyen — DMLS + AI Engineering (from Ch. 11)	Books	Your depth source when the drill book feels shallow
Eugene Yan system-design essays + company engineering blogs of your targets	Blogs	Steal real architectures; cite them in interviews ("Booking published that they...")
interviewing.io and peer mocks (find partners on the Pramp successor platforms or DS Discords)	Mocks	Minimum 6 recorded system-design mocks before Week 24

Chapter 15 — Behavioral Interviews and Staff-Level Leadership

Weeks 4–26, one story per week · Where Staff offers are actually decided

15.1 Why This Exists

At Staff level, behavioral rounds are not a formality — they are the level-calibration mechanism. The question under every question is: "Is this person a force multiplier, or a strong individual contributor we should down-level to Senior?" Your raw material is genuinely good (cross-org Control Tower, platform adopted by teams, \$8M+ impact); what is missing is packaging, quantification, and the deliberate creation — starting now, at Mastercard — of fresh leadership evidence.

15.2 The SAIL Format

Structure every story as SAIL: **Situation** (10% — one crisp scene-setting sentence with stakes), **Action** (30% — what YOU did; "I", not "we"; include the disagreement, the analysis, the decision), **Impact** (40% — numbers, dollars, adoption, and the decision it changed), **Learning** (20% — what changed in how you operate since). Time-box: 2.5–3 minutes spoken. Prepare each story as five bullet points, never a script — scripts die under follow-up questions.

15.3 The Fifteen Stories — Build One Per Week From Week 4

- Identified a problem nobody saw and drove the fix (Innovaccer observability platform is a natural fit).
- Disagreed with a stakeholder and influenced the outcome with data.
- Mentored someone who was struggling (create this at Mastercard NOW if thin — volunteer to onboard/mentor a junior this quarter).
- A real failure and what it changed (pick one with a measurable bad outcome; own it fully).
- Delivered bad news to leadership.
- Balanced short-term delivery vs long-term quality (platform vs feature tension).
- Difficult cross-functional partner (Control Tower era).
- Changed technical direction mid-project.
- Made a call with incomplete information.
- Received hard feedback and acted on it.
- Scaled a solution across teams (Databricks platform adopted by 4 teams — quantify!).
- Said no to something attractive, and why.
- Drove consensus among peers without authority.
- Turned around a failing project.
- Onboarded/developed someone who then exceeded expectations.

Active evidence-creation at Mastercard, starting Week 1: (a) volunteer to mentor one junior DS or new joiner; (b) propose and lead one cross-team initiative (an experimentation guild, an eval-standards doc, a GenAI brown-bag series); (c) write one internal design doc that sets direction. In six months these become true, fresh, verifiable Staff stories — the plan manufactures its own evidence.

15.4 Company Values Mapping

- Amazon: map every story to 2–3 Leadership Principles explicitly (Dive Deep, Ownership, Disagree and Commit, Hire and Develop); rehearse LP vocabulary.
- Google: emphasize ambiguity navigation, intellectual humility, and collaborative influence ("Googleyness").
- Uber/Careem/Talabat: bias for speed, marketplace intuition, ownership under chaos.
- Booking: experimentation humility — stories where the data overturned your (or an executive's) intuition are gold.
- Microsoft: growth mindset — your Learning sections carry extra weight; model-of-learning stories land well.

Chapter 16 — The Application Campaign: Referrals, Sequencing, Negotiation

Weeks 19–26, groundwork from Week 1 · Strategy converts preparation into compensation

16.1 The Market Strategy in One Paragraph

Your prioritization, confirmed by the market analysis: **Primary — UAE** (Careem, Talabat, Property Finder, Revolut UAE): degree-flexible, tax-free income (the fastest legal route to your "earn in crores" goal — a \$140K tax-free package ≈ ■1.2 Cr with no tax drag), active India hiring, and perfect marketplace/recsys/GenAI domain match. **Secondary — India top-tier** (Microsoft, Atlassian, Uber, LinkedIn, Walmart): brand, level, and the internal-transfer route abroad (Microsoft's is the most travelled). **Tertiary — Netherlands** (Booking.com above all): 30% ruling makes the pay real; the degree preference is surmountable with a referral plus your experimentation depth — which, conveniently, is Chapter 1's whole point. **Deprioritize Germany** unless the skilled-worker-without-degree route is confirmed in writing; the Blue Card degree requirement makes it structurally risky for you.

16.2 Networking — Start Week 1, Not Week 19

- Weeks 1–8: rewrite LinkedIn headline ("Senior Applied ML Scientist | Personalization and Ranking | GenAI for Marketing Science"); follow and thoughtfully comment on DS leaders at target companies; publish your first blog post by Week 8.
- Weeks 9–18: 3 new connections per week at target companies with a specific, non-needy message referencing their work; 2–3 informational chats per month; share each project as it ships.
- Weeks 19+: activate referrals company by company, matched to application waves. A referral raises resume-screen pass rates several-fold — treat referral acquisition as a workstream with its own tracker, not an afterthought.

16.3 Application Sequencing

- Wave 1 (Weeks 19–20): 5–6 Tier-2 / practice companies — real interviews are the only true mock; you want your rough loops spent here, not at Careem.
- Wave 2 (Weeks 20–22): UAE primary targets + Microsoft/Atlassian India, all with referrals where possible.
- Wave 3 (Weeks 22–24): Booking.com Amsterdam and remaining Tier-1s, timed so onsites land Weeks 24–26 when you are sharpest.
- Track everything in a simple sheet: company, contact, referral status, stage, next action, dates. Follow up politely at 10 business days.

16.4 Negotiation — Prepare From Week 23, Not After the Offer

- Know the bands before the first recruiter call (Levels.fyi, Glassdoor, community data); never give a number first — give a researched range anchored to the top of band if forced.
- Competing timelines are your only real leverage: sequence interviews so offers can land within the same 2–3 week window; tell recruiters (honestly) that you are in late stages elsewhere.

-
- Negotiate total package: base, bonus, equity, sign-on, relocation, and — for UAE — housing/schooling allowances which are standard and often unlisted.
 - For UAE offers, model post-tax equivalence explicitly against Indian offers; a "lower" Dubai base frequently nets 60–80% more than a Pune package. For NL, model with and without the 30% ruling.
 - Every offer, including the best one: one respectful counter, in writing, with a specific number and a reason. The expected value of asking is enormous; the downside, with professional tone, is nil.

16.5 A Note on the Five-Year Arc

The 26-week plan targets the next role. The next role targets the five-year arc you described: Staff scope → international market → capital accumulation in a strong currency → optionality (research, leadership, or founding). Two habits from this program compound directly into that arc: publishing (your blog posts become a reputation asset) and evaluation rigor (the skill every AI company, including the one you may someday found, is bottlenecked on). Keep both alive after the offer is signed.

PART V

The 26-Week Calendar|Three tracks, every week: Learn · Build · Rehearse

Chapter 17 — The Week-by-Week Master Calendar

Print this chapter. Put it where you can see it. Cross weeks off with a pen.

Phase 1 — Foundations (Weeks 1–8, ~15 hrs/week)

Wk	Learn (5–6 h)	Build (6–8 h)	Rehearse (3–4 h)
1	Kohavi Ch. 1–4; power, sample size, SRM, OEC (Ch. 1)	Sample-size calculator + Cookie Cats A/B analysis	1 DataLemur SQL/day; LinkedIn headline + start Kohavi notes for later RAG corpus
2	Kohavi stats chapters; delta method, CUPED, sequential testing, interference (Ch. 1)	Clustered-experiment + peeking simulations; delta method vs bootstrap	1 SQL + 1 probability/expected-value problem daily
3	Facure Part I; potential outcomes, DAGs, PSM/IPW (Ch. 2)	Lalonde four-estimator comparison	1 SQL/day; 1 ML-theory flashcard session (bias-variance, metrics, calibration)
4	Mixtape/Facure: DiD, synthetic control, IV, RDD (Ch. 2)	Replicate California Prop 99 with placebos	First SAIL story written + spoken; 1 SQL/day
5	Karpathy Zero-to-Hero pt 1–2; PyTorch basics (Ch. 3) + Eugene Yan recsys + YouTube paper (Ch. 4)	MLP training loop from scratch; start Two-Tower on MovieLens	1 SQL/day; story #2
6	LTR: Burges paper, NDCG, position bias, IPS (Ch. 5)	Finish Two-Tower + FAISS; LambdaMART on MSLR; position-bias simulation	Advanced SQL (sessionization, funnels); story #3
7	Transformers: Illustrated Transformer → Karpathy GPT video (Ch. 6)	nanoGPT-style model + attention visualizations; parameter-count exercise	Matrix-calculus/backprop refresh; story #4
8	RAG fundamentals: DL.AI short course + Pinecone guides + RAGAS quickstart (Ch. 7)	Mini-RAG over your own notes with golden set + RAGAS	First mock: 45-min Python/pandas screen with a peer; publish blog post #1

Phase 1 exit criteria: all Chapter 1–7 checkpoints pass; Two-Tower and nanoGPT repos exist; 4 SAIL stories drafted; 60+ SQL problems solved; 1 blog post live. If any fails, take a catch-up week before Phase 2 — Phase 2 has slack for exactly one such pause.

Phase 2 — GenAI Depth + Portfolio (Weeks 9–18, ~18 hrs/week)

Wk	Learn	Build	Rehearse
9	LoRA/QLoRA: PEFT docs, LoRA paper, DL.AI finetuning course (Ch. 8)	Project 3 start: baseline recommender + zero-shot explanations; warm-up QLoRA run	2 SQL/day begins; rewrite first two resume bullets with new vocabulary
10	Evaluation: Eugene Yan evals, judge biases, human-eval design (Ch. 9)	Project 3: curate pairs, QLoRA training, rank ablation	Recsys system-design drill #1 (Talabat feed), recorded
11	Agents: Anthropic "Building Effective Agents", ReAct paper, LangGraph academy (Ch. 10)	Project 3: human eval + judge calibration, ship. Project 1 start: simulator + raw ReAct agent	Stories #5–6; 1 SQL/day maintenance

Wk	Learn	Build	Rehearse
12	LangGraph deep work: state, routing, interrupts (Ch. 10)	Project 1: four agents with budgets and approval gates	Product-sense mock (metric design + root cause); blog post #2 (position bias)
13	Marketing science: attribution vs incrementality, geo-designs, MMM overview (Ch. 2/10 bridge)	Project 1: evaluation harness with CIs, failure appendix, Streamlit, ship	Causal-inference Q&A drill; story #7
14	RAG evaluation deep dive: RAGAS metrics, golden sets, error taxonomies (Ch. 9)	Project 2 start: ingestion, chunking comparison, retrieval-only eval	SQL + experimentation combo drills (Booking style)
15	Vector DBs and hybrid retrieval internals; DMLS begins (Ch. 7/11)	Project 2: hybrid + rerank + citations + full RAGAS report	System-design drill #2 (semantic search), recorded
16	Production GenAI: vLLM concepts, caching, cost model (Ch. 11)	Project 2: FastAPI + latency/cost benchmark, ship. Project 4 start: simulator + geo-matching	ML-theory deep-dive mock with a peer
17	Heterogeneous effects: meta-learners, causal forests, Qini (Ch. 2)	Project 4: synthetic control + DiD + uplift module	Leadership stories #8–10; Staff-level "setting direction" narratives
18	DMLS continued; monitoring and drift (Ch. 11)	Project 4 ship. Polish week: all READMEs, diagrams, videos, pinned profile; blog post #3	Full self-run mock loop (SQL + design + behavioral in one day)

Phase 2 exit criteria: four shipped repos meeting the six-artifact standard; resume fully rewritten; 10 stories rehearsed; 3 blog posts; LinkedIn network at target companies actively warm.

Phase 3 — Interviews and the Campaign (Weeks 19–26)

Wk	Learn / Review	Campaign	Rehearse
19	Review Ch. 1–2 edge cases; assemble a personal "formula and framework" sheet	Resume final pass; application tracker built; referral asks batch #1	2 mocks: SQL screen + ML system design
20	Company research: Careem, Talabat, Booking, Microsoft loops and products	Wave 1 applications (5–6 practice-tier); 10 outreach messages	2 mocks: design + behavioral, recorded and reviewed
21	Target-company case prep: their metrics, their public architectures	Wave 2: UAE targets + Microsoft/Atlassian with referrals; follow-ups	2 mocks: product sense + leadership; STAR/SAIL timing discipline
22	GenAI system-design review (RAG + agents patterns)	Wave 3 prep; portfolio 5-minute pitch polished	Mock AI-system design; portfolio walkthrough to a friend who asks hostile questions
23	DSA focus block: hash maps, two pointers, sliding window, BFS/DFS	Wave 3: Booking + remaining Tier-1; negotiation research (bands, comparables)	1 coding mock + 1 full-loop simulation
24	Review weak areas flagged by mocks; behavioral polish	Referral activation completed; interview scheduling to cluster offers	Final full mock loop (4 rounds in one day)
25	Light review only; company-specific "why us" answers	Onsite logistics; offer-timeline choreography with recruiters	Daily 20-min story run-throughs; sleep discipline
26	Deep familiarity pass over all four projects — every decision, every trade-off	Onsites; counter-offers in writing; decision	Rest between rounds; light whiteboard warm-ups only

The Weekly Rhythm (Template)

Slot	Hours	What happens
Weekday mornings or lunch (×5)	3–4	SQL/coding problems and flashcards — small, daily, non-negotiable
Weekday evenings (×3–4)	6–8	Build track: project and implementation work in 90–120 min deep blocks
Saturday morning	3	Learn track: primary book/course of the week
Sunday morning	2–3	Learn track continued + the Feynman whiteboard ritual (record it)
Sunday evening	1	Review the week, plan the next, update tracker, one LinkedIn touch

APPENDICES

Reference Material|

Appendix A — The Master Resource Library

Books (buy or bookmark in Week 1)

- Kohavi, Tang, Xu — Trustworthy Online Controlled Experiments (experimentation bible).
- Facure — Causal Inference for the Brave and True (free); Cunningham — The Mixtape (free); Huntington-Klein — The Effect (free).
- Chip Huyen — Designing Machine Learning Systems, and AI Engineering.
- Aminian and Xu — Machine Learning System Design Interview.
- Singh and Huo — Ace the Data Science Interview.
- Raschka — Build a Large Language Model (From Scratch) (optional depth).

Courses and video (all free)

- Karpathy — Neural Networks: Zero to Hero (incl. "Let's build GPT").
- 3Blue1Brown — Neural networks and attention series.
- Hugging Face NLP Course; LangChain Academy — Intro to LangGraph.
- DeepLearning.AI short courses: Finetuning LLMs; Building and Evaluating Advanced RAG; AI Agents in LangGraph.
- Brady Neal — Introduction to Causal Inference; Google — Recommendation Systems course; Udacity A/B Testing.

Papers (the 12 that matter for you)

- Attention Is All You Need (2017) · LoRA (2021) · QLoRA (2023) · ReAct (2022).
- YouTube Recommendations (Covington 2016) · DeepFM · DCN-v2 · DIN · MMoE.
- Burges — From RankNet to LambdaRank to LambdaMART · Joachims unbiased LTR · MT-Bench LLM-as-judge (2023).

Blogs to follow weekly

- Eugene Yan · Chip Huyen · Sebastian Raschka · Hamel Husain · Lilian Weng.
- Engineering blogs: Booking, Uber, Airbnb, Netflix, Spotify, Meta AI, Anthropic engineering, Pinecone learning center.

Practice platforms

- DataLemur, LeetCode (SQL 50 + NeetCode subset), StrataScratch, interviewing.io for mocks.

Libraries you will touch

- PyTorch, Hugging Face (transformers, PEFT, TRL, datasets), Unsloth, LightGBM, FAISS, ChromaDB/Qdrant, LangChain/LangGraph, RAGAS, EconML, DoWhy, CausalML, statsmodels, Feast, FastAPI, Streamlit, wandb, vLLM (concepts).

Appendix B — The Resume Rewrite Checklist (Execute in Week 19)

- Summary: 3 lines max — years + domains (personalization, experimentation, GenAI for marketing science), scale (10M+ households, 1B+ pairs), quantified impact (\$8M+), leadership scope. Kill "proven track record"; fix stray capitalization ("ML PLatform" typo included).
- Skills: replace Wide & Deep/LSTM/CART framing with: Two-Tower and multi-task ranking, learning-to-rank, causal inference and experimentation, PyTorch, RAG (hybrid retrieval, RAGAS), LoRA/QLoRA fine-tuning, LangGraph agents, vLLM/FastAPI serving. Drop "Selective GenAI Exposure" — the word "selective" is an apology; rename the section "GenAI and LLM Systems."
- Every bullet: metric + baseline/context + your specific action + business consequence. If a bullet has no number, it earns one or it dies.
- Rewrite the Innovaccer GPT-4o bullet as a RAG system with evaluation methodology (truthfully — you will now have the vocabulary for what you actually built).
- Albertsons bullet: add experiment design context (users, duration, guardrails) you can defend.
- Add a "Selected Projects" section with 3 of the 4 portfolio projects, one line each with the headline result.
- Accenture: compress to two lines. Publications: remove the 2016 RFID paper. Awards: keep one line, add selectivity context if known.
- Add Mastercard role with 2–3 bullets by Week 19 — actively engineer at least one experimentation and one GenAI deliverable at work in Months 1–4 so these bullets are real and current.
- Pass the 30-second test: give it to two friends; ask what level and specialty they'd guess. If the answer is not "senior/staff, personalization + GenAI," iterate.

Appendix C — Monthly Checkpoints and Self-Assessment

Month	You should now be able to...	Red flags meaning you must adjust
1	Design and critique A/B tests fluently; run four causal estimators on real data; explain assumptions of each	Still fuzzy on p-values/power; skipped the builds and only watched videos
2	Write PyTorch training loops cold; whiteboard four-stage recsys; explain attention with math; basic RAG built	Two-Tower or nanoGPT repo not finished; SQL habit broken more than 5 days
3	Fine-tune with QLoRA and defend the choices; Projects 3 and 1 shipped to the six-artifact standard	Projects "90% done" (i.e., not done); no human-eval component; zero blog posts
4	Full RAG platform with RAGAS report and serving benchmarks; 8+ stories rehearsed	Evaluation reports missing; networking not started ("I'll do it after the projects")
5	All four projects shipped; resume rewritten; system-design spine automatic; Wave 1–2 applications out	Fewer than 4 recorded mocks; no referrals in motion; tracker empty
6	Interviewing at full strength; offers clustering; negotiating in writing	Applications went out but stories untimed; accepting the first number offered

Appendix D — The Forgotten-Math Refresher (Slot Into Rehearse Time, Weeks 3–7)

- Expected value and variance manipulations; common distributions and their means/variances (Bernoulli, binomial, Poisson, normal, exponential).
- Central Limit Theorem: conditions, and the violations that matter for A/B tests (heavy tails, dependence).
- MLE vs MAP; where regularization = a prior.
- Bias-variance decomposition — derive it once.
- Gradient boosting internals: the functional-gradient view; what XGBoost adds (second-order, regularized splits).
- Matrix calculus for backprop: gradients of linear layers and softmax-cross-entropy (done properly in Chapter 3/6 builds).
- Bayes theorem fluency with medical-test style problems (they still get asked).
- Calibration: reliability curves, Platt/isotonic — connects to your AUPRC story.

A Closing Word

Twenty-six weeks from now, you will not be a different person — you will be the same builder you already are, finally legible to the market at your true level. The degree you do not have is a fixed constraint; everything else in this book is a variable you control. Control the variables. Do the weeks. And when the Careem or Booking offer is on the table and you are drafting the counter, re-read the letter at the front of this book and notice how far the fuel carried you.

Now go start Week 1. The sample-size calculator is waiting.